SE PROJECT · 2026 · FINAL DELIVERY

Smart *E-Ticketing* System

Software Engineering — Course Project

Demonstrating Agile/Scrum, Clean Code, SOLID, GoF Design Patterns,
automated testing, CI/CD, Docker, and live deployment.

Team: Mahmoud (PO & Scrum Master) · Mohammed (SE) · Esraa (SE) · Ali (Backend Dev) · Amr (Frontend Dev) · Seif (QA)

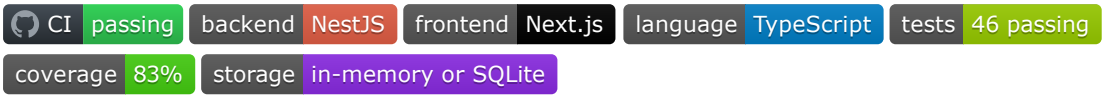
Supervisor: Dr. Ihab Ramadan

Live frontend: https://smart-e-ticket.vercel.app
Live backend: https://smart-eticketing-backend-production-0222.up.railway.app/api-docs
Source: https://github.com/MahmoudSayed0/Smart-E-ticket-PR

Table of Contents

Part 1 — Project Overview

> A full-stack **Software Engineering course project** demonstrating Agile/Scrum, Clean Code, SOLID principles, and three GoF design patterns: **Repository**, **Factory**, and **Strategy**.



🌐 Live Deployment

	URL	Stack
Frontend	smart-e-ticket.vercel.app	Next.js 15 on **Vercel**
Backend API	smart-eticketing-backend-production-0222.up.railway.app	NestJS in **Docker** on **Railway** (SQLite repository binding)
Swagger UI	`/api-docs`	Auto-generated OpenAPI
GitHub Actions	Workflow runs	Lint · test · build on every push

> ⚠️ The Railway free tier sleeps the backend after 15 min idle. Wake it by hitting the Swagger URL once before testing the live frontend.

🚢 CI/CD

Pipeline	Trigger	What it runs
CI (`.github/workflows/ci.yml`)	every push + every PR	install → lint → unit tests with coverage → build (backend + frontend in parallel)
CD — frontend	push to `main`	auto-deploys to **Vercel** (project Root Directory = `frontend/`). `NEXT_PUBLIC_API_BASE` env var points at the Railway backend URL.
CD — backend	push to `main`	builds `backend/Dockerfile` (Node 22, build tools for `better-sqlite3` native binding) and ships the container to **Railway**. `DB_DRIVER=sqlite` is set in the image so the SQLite repository binding kicks in at boot.

Pluggable storage — the Repository pattern in action

Storage is decided at boot from the `DB_DRIVER` environment variable:

```
// backend/src/app.module.ts
const useSqlite = process.env.DB_DRIVER === 'sqlite';
{ provide: EVENT_REPOSITORY,  useClass: useSqlite ? SqliteEventRepository  : InMemoryEven
{ provide: TICKET_REPOSITORY, useClass: useSqlite ? SqliteTicketRepository : InMemoryTick
```

Environment	`DB_DRIVER`	Backing store
Local dev / `npm test`	unset	In-memory `Map<>` (fast, no setup)
Production (Railway, Docker)	`sqlite`	`better-sqlite3` writing to `data/eticketing.db` inside the container

`grep -r 'Sqlite\|InMemory' backend/src/service/` returns **zero matches** — services depend only on the interfaces. That is the entire point of the pattern, and you can witness it live by hitting the deployed URL.

📋 Overview

The Smart E-Ticketing System is an **MVP web application** where administrators create events (concerts, conferences, sports) and generate unique one-time-use tickets for them. Ticket holders view their tickets via a unique link, see a countdown timer until the event starts, and redeem the ticket at the venue. The system prevents double-redemption and blocks operations on expired events.

This project is intentionally an **academic exercise** — the goal is to practice software engineering disciplines (SRS, UML, SOLID, patterns, tests, Agile), not to ship a commercial product.

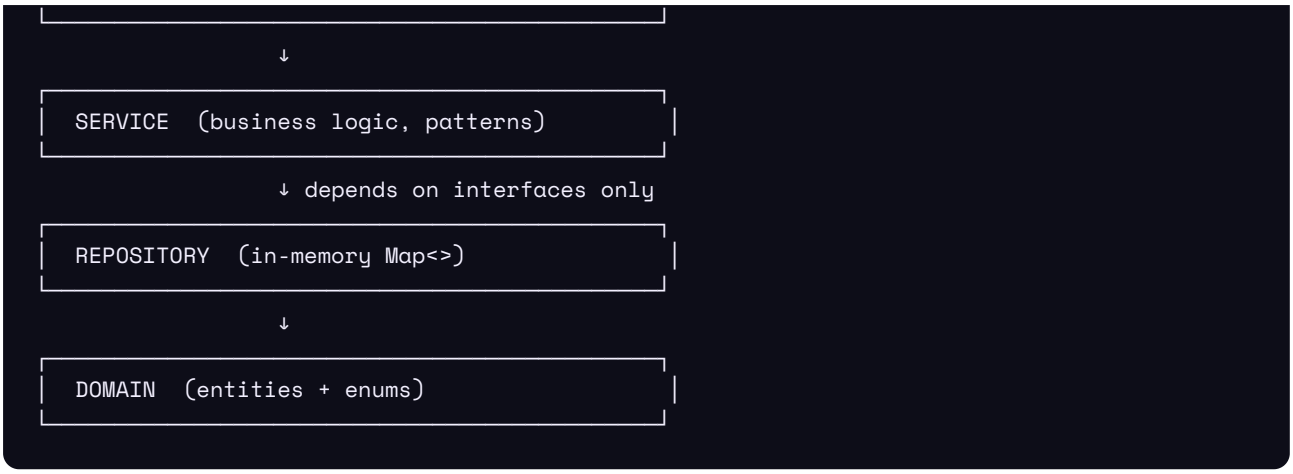
🎯 Features

- ✅ Create events (Concert, Conference, Sports — polymorphic via Factory pattern)
- ✅ Generate one-time-use tickets with unique codes (Strategy pattern)
- ✅ View tickets via unique URL with live QR code and 3D tilt animation
- ✅ Redeem tickets at the venue (one-time only)
- ✅ Live countdown timer — event expires at `eventDate`
- ✅ Backend enforces: no redemption after expiry, no generation on expired events, no double-redemption
- ✅ In-memory repository behind an interface (swappable to a real DB without touching services)
- ✅ Swagger UI auto-generated at `/api-docs`
- ✅ Responsive dashboard with sidebar, KPI cards, bar chart, and sticky create form

🏗️ Architecture

Four-layer **Layered Architecture** with strict dependency direction (top → bottom):

```
┌──────────────────────────────────────────┐
│  PRESENTATION  (Next.js pages, React)    │
└──────────────────────────────────────────┘
                  ↓ HTTP
┌──────────────────────────────────────────┐
│  CONTROLLER  (NestJS REST endpoints)     │
```



Every folder in `backend/src/` maps 1:1 to a layer. See `docs/ARCHITECTURE.md` for the full walkthrough.

🧩 Design Patterns Implemented

Pattern	Location	Purpose
Repository	`backend/src/repository/`	Abstracts data access behind `IEventRepository` and `ITicketRepository`, enabling in-memory now and PostgreSQL later with zero service changes
Factory	`backend/src/factory/event.factory.ts`	Creates polymorphic `ConcertEvent`, `ConferenceEvent`, or `SportsEvent` from a single `CreateEventDto`
Strategy	`backend/src/strategy/`	Three interchangeable ticket-code algorithms (`UuidCodeStrategy`, `ShortCodeStrategy`, `NumericCodeStrategy`) selected via DI

🛠️ Tech Stack

Layer	Tool	Why
Backend	NestJS 10 + TypeScript	Built-in DI container makes SOLID principles visible and enforceable
Frontend	Next.js 15 + React 19 + TypeScript	Modern web app, App Router, server/client split
Styling	Tailwind CSS 3 + Soft-pop theme (tweakcn) + shadcn-style dashboard	Unique look, fast build
Icons	Lucide React	Consistent, lightweight icon set
Charts	Recharts	Bar chart on the admin dashboard
3D Animation	Motion (Framer Motion)	TiltedCard wrapper on the ticket page
Testing	Jest + NestJS Testing utilities	46 unit tests, 83% coverage
API Docs	@nestjs/swagger	Auto-generated OpenAPI spec at `/api-docs`

Version Control	Git + GitHub	Feature branches + PRs
Agile Tracking	Notion	Sprint plans, backlog, retrospectives

🚀 Getting Started

Prerequisites

- Node.js 20 or newer
- npm (ships with Node)

Setup

```
## 1. Clone the repo
git clone <repo-url> smart-eticketing
cd smart-eticketing

## 2. Install backend dependencies
cd backend
npm install

## 3. Install frontend dependencies
cd ../frontend
npm install
```

Run in development

Open **two terminals**:

```
## Terminal 1 — backend (port 4000)
cd backend
npm run start:dev
## → http://localhost:4000
## → Swagger docs at http://localhost:4000/api-docs
```

```
## Terminal 2 — frontend (port 3000)
cd frontend
npm run dev
## → http://localhost:3000
## → Admin dashboard at http://localhost:3000/admin
```

Run the tests

```
cd backend
npm test            # run all tests
npm run test:cov    # with coverage report
```

Expected output: **46 tests passing, 8 suites green, ~83% line coverage**.

📂 Project Structure

```
smart-eticketing/
├── backend/                        NestJS backend
│   ├── src/
│   │   ├── domain/                 Entities (Event, Ticket) + enums
│   │   ├── repository/             Repository pattern (interfaces + in-memory impls)
│   │   ├── factory/                Factory pattern (EventFactory)
│   │   ├── strategy/               Strategy pattern (3 ticket code strategies)
│   │   ├── service/                Business logic (EventService, TicketService)
│   │   ├── controller/             REST endpoints
│   │   ├── dto/                    Data Transfer Objects + class-validator rules
│   │   ├── app.module.ts           DI wiring (binds interfaces → implementations)
│   │   └── main.ts                 Bootstrap + Swagger setup
│   ├── package.json
│   └── tsconfig.json
│
├── frontend/                       Next.js frontend
│   ├── app/
│   │   ├── page.tsx                Landing page
│   │   ├── admin/page.tsx          Admin dashboard (sidebar + KPIs + chart + events)
│   │   ├── tickets/[id]/page.tsx   Ticket detail (QR + countdown + redeem)
│   │   ├── lib/
│   │   │   ├── api.ts              Typed fetch client
│   │   │   ├── useCountdown.ts     React hook for live countdown
│   │   │   └── utils.ts            cn() class helper
│   │   ├── layout.tsx              Root layout with DM Sans + Space Mono
│   │   └── globals.css             Soft-pop theme CSS variables
│   ├── components/
│   │   ├── AppSidebar.tsx          Dashboard sidebar
│   │   ├── SiteHeader.tsx          Sticky header with breadcrumb + search
│   │   ├── TicketsChart.tsx        Recharts bar chart
│   │   ├── AnimatedTicket.tsx      Ticket card with cut-outs + QR + details
│   │   ├── TiltedCard.tsx          3D mouse-follow wrapper
│   │   └── Countdown.tsx           Live countdown (badge + large variants)
│   └── tailwind.config.ts
│
├── docs/                           All project documentation
│   ├── SRS.md                      Software Requirements Specification
│   ├── ARCHITECTURE.md             Architecture & design patterns walkthrough
│   ├── TEST_CASES.md               Manual test cases (QA-owned)
│   ├── PRESENTATION.md             Slide outline + speaker notes
│   ├── DEMO_SCRIPT.md              Step-by-step live demo script
│   └── uml/                        PlantUML source files
│       ├── use-case-diagram.puml
│       ├── class-diagram.puml
```

```
│       ├── sequence-generate-ticket.puml
│       ├── sequence-redeem-ticket.puml
│       └── activity-ticket-lifecycle.puml
│
└── README.md                       (this file)
```

🧪 Testing

The backend has **46 unit tests** across **8 test suites**:

Suite	Tests	Coverage
`domain/event.entity.spec.ts`	8	100%
`domain/ticket.entity.spec.ts`	3	100%
`repository/in-memory-event.repository.spec.ts`	5	100%
`repository/in-memory-ticket.repository.spec.ts`	4	100%
`strategy/strategies.spec.ts`	5	100%
`factory/event.factory.spec.ts`	5	93%
`service/event.service.spec.ts`	5	100%
`service/ticket.service.spec.ts`	11	100%

Every service test **mocks its repository and strategy dependencies**, which proves that our Dependency Inversion is correctly wired — if the code weren't depending on interfaces, mocking would not be possible in one line.

See `docs/TEST_CASES.md` for the human-readable test case document used for manual QA.

📖 API Documentation

When the backend is running, open **http://localhost:4000/api-docs** for the interactive Swagger UI.

Endpoints

Method	Path	Description
`POST`	`/events`	Create a new event
`GET`	`/events`	List all events
`GET`	`/events/:id`	Get a single event
`POST`	`/events/:eventId/tickets`	Generate a ticket
`GET`	`/events/:eventId/tickets`	List tickets for an event
`GET`	`/tickets/:id`	View a single ticket
`POST`	`/tickets/:id/redeem`	Redeem a ticket

All responses are JSON. All request DTOs are validated with `class-validator`.

👥 Team

Member	Scrum Role	Primary Deliverable
Mahmoud	Product Owner & Scrum Master	SRS, scope, backlog grooming, sprint rituals, Notion workspace, Definition of Done
Mohammed	Software Engineer	Architecture, UML (use case + class), design rationale, layered architecture decisions
Esraa	Software Engineer	Sequence + activity diagrams, schema, database-swap design, SOLID review
Ali	Backend Developer	NestJS backend, the 3 GoF patterns in code (Repository · Factory · Strategy), Swagger, DI module wiring
Amr	Frontend Developer	Next.js 15 frontend, `/admin` dashboard, `/tickets/[id]` page, AnimatedTicket, soft-pop theme, SOLID audit, live demo
Seif	QA / Test Engineer	Jest unit tests, coverage reports, test cases document, mocking discipline

> Supervised by **Dr. Ihab Ramadan** · Software Engineering, 2026

📅 Agile Process

Four one-week sprints were simulated and tracked in Notion:

1. **Sprint 1** — Requirements & Design (SRS, UML, architecture)
2. **Sprint 2** — Domain & Repository layer (entities + in-memory repos + 20 tests)
3. **Sprint 3** — Services, Patterns, REST API (Factory + Strategy + controllers + 26 tests)
4. **Sprint 4** — Frontend, Polish & Demo (Next.js, presentation, expiry enforcement)

Every sprint included Sprint Planning, Daily Standups, Sprint Review, and a Retrospective. See the Notion workspace for the full history.

📜 License

Academic / Educational — not for commercial use.

Built with 💙 by the Smart E-Ticketing team for the Software Engineering course.

Part 2 — Software Requirements Specification

Smart E-Ticketing System

Version: 1.1 **Date:** April 2026 **Prepared by:** [Team Name] **Course:** Software Engineering **Owner:** Product Owner (Mahmoud)

---

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements of the **Smart E-Ticketing System**, a web-based application that enables event administrators to create events, generate one-time-use tickets, and allows ticket holders to view and redeem their tickets through a unique link.

The purpose of this SRS is to serve as a contract between the development team and stakeholders, defining the scope, behavior, and constraints of the system before development begins.

1.2 Scope

The Smart E-Ticketing System is an **MVP (Minimum Viable Product)** developed as an academic exercise to practice core software engineering principles: Agile methodology, SOLID design, design patterns, clean code, unit testing, and version control.

In Scope:

- Creation and management of general-admission events by an administrator
- Generation of unique, one-time-use tickets tied to events
- Public viewing of a ticket via a unique URL
- Ticket redemption (marking as "used") at the venue gate
- Prevention of reusing an already-redeemed ticket
- **Prevention of operations on expired events (added v1.1)**
- In-memory data persistence abstracted behind the Repository pattern

Out of Scope (intentionally excluded for the MVP):

- Payment processing or financial transactions
- Seat selection or real-time seat locking
- User registration, login, or password management
- Email or SMS notifications
- Refund, cancellation, or ticket transfer workflows
- Multi-language or accessibility localization
- Mobile native application
- Persistent database (PostgreSQL, MySQL, etc.) — simulated via in-memory repository

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Event	A scheduled happening (concert, conference, match) for which tickets are sold
Ticket	A unique, one-time-use entry pass associated with an event
Ticket Code	A unique alphanumeric identifier printed/shown on the ticket
Redemption	The act of using a ticket to gain entry; once redeemed, the ticket becomes invalid
Expired Event	An event whose `eventDate` is in the past; the system rejects ticket operations on expired events
Admin	User with privileges to create events and generate tickets
Ticket Holder	End user who possesses a ticket and may redeem it
MVP	Minimum Viable Product
SRS	Software Requirements Specification
API	Application Programming Interface
DTO	Data Transfer Object
SOLID	Five object-oriented design principles (SRP, OCP, LSP, ISP, DIP)

1.4 References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications
- Course Guidelines: *Software Engineering Project Guidelines*
- Gang of Four, *Design Patterns: Elements of Reusable Object-Oriented Software*

---

2. Overall Description

2.1 Product Perspective

The Smart E-Ticketing System is a **standalone web application** consisting of:

- A **backend REST API** (NestJS + TypeScript) exposing endpoints for event and ticket management
- A **frontend web client** (Next.js + TypeScript) providing the admin dashboard and public ticket page
- An **in-memory data store** accessed exclusively through Repository interfaces, designed to be swappable with a relational database in the future without affecting business logic

The system has no dependencies on external services.

2.2 Product Functions

At a high level, the system shall:

1. Allow an administrator to create new events
2. Allow an administrator to generate one-time-use tickets for an event
3. Allow an administrator to view a list of all events and their ticket counts
4. Allow a ticket holder to view the details of a specific ticket via a unique URL
5. Allow a ticket holder (or gate staff) to redeem a ticket, marking it as used

6. Prevent the redemption of an already-used ticket
7. Prevent the generation of tickets beyond the event's defined capacity
8. Prevent the generation and redemption of tickets on expired events (v1.1)

2.3 User Classes and Characteristics

User Class	Description	Technical Skill	Frequency of Use
Administrator	Event organizer who creates events and generates tickets	Basic web literacy	Daily during event setup
Ticket Holder	End user who receives a ticket link and views/redeems the ticket	Minimal	Once per ticket
Gate Staff	Person at the venue entrance who validates tickets	Minimal	Repeatedly during event

2.4 Operating Environment

- **Client side**: Any modern web browser (Chrome, Firefox, Safari, Edge) on desktop or mobile
- **Server side**: Node.js 20+ runtime; runs locally on `localhost:3000` (frontend) and `localhost:4000` (backend) for the MVP demo
- **Development OS**: macOS, Windows, or Linux
- **Network**: Local HTTP (no HTTPS required for MVP)

2.5 Design and Implementation Constraints

- **CON-01**: The system **must** use at least 3 design patterns from the following list: Repository, Factory, Strategy, Observer
- **CON-02**: The system **must** follow SOLID principles throughout the codebase
- **CON-03**: Code **must** follow Clean Code guidelines: meaningful names, small functions, no duplication
- **CON-04**: The data layer **must** be implemented in-memory but architected behind Repository interfaces, such that swapping to a real relational database would require no changes to the service layer
- **CON-05**: All business logic **must** be covered by unit tests using Jest
- **CON-06**: All source code **must** be managed in Git with feature branches and pull requests
- **CON-07**: The project **must** be delivered using Agile/Scrum methodology across 4 sprints

2.6 Assumptions and Dependencies

- **ASM-01**: The administrator is trusted; no authentication is required for the MVP (single-admin assumption)
- **ASM-02**: Ticket holders receive the ticket URL out-of-band (manually shared); the system does not send emails
- **ASM-03**: The demo environment is a single machine running both frontend and backend locally
- **ASM-04**: Only one administrator operates the system at a time (no multi-admin conflicts)

---

3. Functional Requirements

3.1 FR-01: Create Event

Priority: High **Description**: The system shall allow an administrator to create a new event by providing the event name, description, venue, date, and total ticket capacity. **Input**: Event name (3–100 chars), description (≤500 chars), venue, event date (ISO date, must be in the future), total capacity (≥1), event type (CONCERT | CONFERENCE | SPORTS) **Processing**: System validates input, assigns a unique event ID, initializes `remainingCapacity` equal to `totalCapacity`, timestamps creation, persists via `IEventRepository` **Output**: Confirmation with the newly created event ID and full event details **Error Cases**:

- E-01.1: Any required field missing → HTTP 400 with validation message
- E-01.2: Event date in the past → HTTP 400 "Event date must be in the future"
- E-01.3: Capacity ≤ 0 → HTTP 400 "Capacity must be at least 1"

3.2 FR-02: Generate Ticket

Priority: High **Description**: The system shall allow an administrator to generate a new one-time-use ticket for an existing event. **Input**: Event ID (UUID) **Processing**: System locates the event via `IEventRepository`; **verifies the event has not expired**; checks remaining capacity; generates a unique ticket code using an `ITicketCodeStrategy`; creates a ticket with status `VALID`; decrements remaining capacity; persists via `ITicketRepository` **Output**: The generated ticket including its unique code and redemption URL **Error Cases**:

- E-02.1: Event ID not found → HTTP 404 "Event not found"
- E-02.2: Event capacity exhausted → HTTP 409 "No tickets remaining for this event"
- **E-02.3: Event has already ended → HTTP 409 "Event has already ended" (v1.1)**

3.3 FR-03: List Events

Priority: Medium **Description**: The system shall allow an administrator to view all events with their details and remaining ticket counts. **Input**: None **Processing**: System fetches all events via `IEventRepository.findAll()` **Output**: Array of events with ID, name, venue, date, total capacity, remaining capacity, and expiry status **Error Cases**: None (empty array returned if no events exist)

3.4 FR-04: View Ticket

Priority: High **Description**: The system shall allow any user with a valid ticket URL to view the ticket's details. **Input**: Ticket ID (UUID) in the URL path **Processing**: System fetches the ticket via `ITicketRepository.findById()`; fetches the associated event **Output**: Ticket details including event name, venue, date, ticket code, status (VALID / USED / EXPIRED), countdown until event start, and a QR code representation of the ticket code **Error Cases**:

- E-04.1: Ticket ID not found → HTTP 404 "Ticket not found"

3.5 FR-05: Redeem Ticket

Priority: High **Description**: The system shall allow a ticket holder or gate staff to redeem a valid ticket, marking it as used and preventing future redemptions. **Input**: Ticket ID (UUID) **Processing**: System locates the ticket; verifies the ticket status is `VALID`; **verifies the associated event has not expired**; updates status to `USED`; records the redemption timestamp; persists the change **Output**: Confirmation that the ticket has been redeemed **Error Cases**:

- E-05.1: Ticket ID not found → HTTP 404 "Ticket not found"
- E-05.2: Ticket already used → HTTP 409 "This ticket has already been redeemed"
- **E-05.3: Event has ended → HTTP 409 "Event has ended. This ticket can no longer be redeemed" (v1.1)**

3.6 FR-06: Prevent Overselling

Priority: High **Description**: The system shall never allow more tickets to be generated than the event's total capacity. **Implementation**: Enforced atomically in `TicketService.generateTicket()` by checking and decrementing remaining capacity in the same operation, protected by the repository's thread-safe operations.

3.7 FR-07: Prevent Ticket Reuse

Priority: High **Description**: The system shall guarantee that once a ticket has been redeemed, it cannot be redeemed again. **Implementation**: Enforced in `TicketService.redeemTicket()` by checking the ticket's status before performing the update.

3.8 FR-08: Enforce Event Expiry (v1.1)

Priority: High **Description**: The system shall prevent the generation and redemption of tickets for events whose `eventDate` is in the past. An event becomes "expired" the moment the current time crosses `eventDate`. **Implementation**: A new method `Event.isExpired()` on the domain entity returns true when `eventDate <= Date.now()`. Both `TicketService.generateTicket` and `TicketService.redeemTicket` call `isExpired()` before performing their operation and throw `ConflictException` if the event has expired. **UI Behaviour**: The frontend displays a live countdown timer via `useCountdown` hook, disables the Generate button, and shows an "EXPIRED" state with the `Clock` icon on the ticket page when the event has ended. **Rationale**: Added in Sprint 4 after the team noticed that the previous design allowed ticket generation and redemption on past-dated events, which does not match real-world behavior.

3.9 FR-09: List Tickets for an Event

Priority: Medium **Description**: The system shall allow an administrator to view all tickets generated for a specific event. **Input**: Event ID (UUID) **Processing**: `TicketService.getTicketsForEvent()` calls `ITicketRepository.findByEventId()` **Output**: Array of tickets with id, code, status, created date, and redemption timestamp **Error Cases**: None (empty array returned if the event has no tickets)

---

4. External Interface Requirements

4.1 User Interfaces

The frontend provides **two primary pages**:

4.1.1 Admin Dashboard (`/admin`)

- Sidebar navigation (Dashboard, Events, Tickets, Analytics, Settings)
- Sticky header with breadcrumb and search
- Four KPI cards: Total Events, Tickets Issued, Valid, Redeemed
- Bar chart: Tickets per Event (valid vs redeemed)
- Create event form (left column, sticky)
- Events list (right column) with capacity bar, countdown, and expandable ticket list per event
- Each ticket row shows Copy and Open actions (disabled for redeemed tickets)

4.1.2 Ticket Page (`/tickets/[id]`)

- Animated ticket card with cut-out circles and dashed separators
- Header: icon (CheckCircle2 / BadgeCheck / Clock) + E-TICKET label + status badge
- Event name and type
- Details rows (Venue, Date, Ticket ID, Redeemed at)

- Live countdown ("Ends in X")
- QR code with the ticket code displayed below
- Redeem button (shown only for VALID tickets)
- 3D tilt effect on mouse hover

4.2 Software Interfaces

The backend exposes a REST API under `http://localhost:4000` with the following endpoints:

Method	Endpoint	Description	Maps to
POST	`/events`	Create a new event	FR-01
GET	`/events`	List all events	FR-03
GET	`/events/:id`	Get a single event	FR-03
POST	`/events/:id/tickets`	Generate a ticket for an event	FR-02, FR-06, FR-08
GET	`/events/:id/tickets`	List all tickets for an event	FR-09
GET	`/tickets/:id`	View ticket details	FR-04
POST	`/tickets/:id/redeem`	Redeem a ticket	FR-05, FR-07, FR-08

All responses are JSON. Interactive API documentation is available at `http://localhost:4000/api-docs` (Swagger UI).

4.3 Communications Interfaces

- HTTP/1.1 over TCP on localhost
- JSON request/response bodies (`Content-Type: application/json`)
- CORS enabled between frontend (`:3000`) and backend (`:4000`)

---

5. Non-Functional Requirements

5.1 Performance

- **NFR-01**: API responses shall complete in under 100ms for any single-entity operation
- **NFR-02**: The admin dashboard shall render its initial view in under 1 second on a modern browser

5.2 Usability

- **NFR-03**: The admin shall be able to create an event in no more than 7 form fields and 1 button click
- **NFR-04**: The ticket page shall be readable on screens as small as 320px wide (mobile-friendly)
- **NFR-05**: Error messages shall be displayed in plain language, not as raw stack traces

5.3 Maintainability

- **NFR-06**: The codebase shall achieve ≥70% unit-test coverage on the service layer (actual: 100%)
- **NFR-07**: All classes shall adhere to SOLID principles, verifiable by code review
- **NFR-08**: No method shall exceed 30 lines of code; no file shall exceed 300 lines

- **NFR-09**: The data-access layer shall be replaceable by implementing the repository interfaces without modifying service or controller code

5.4 Reliability

- **NFR-10**: The system shall guarantee that a ticket can never be redeemed twice
- **NFR-11**: The system shall guarantee that the number of generated tickets never exceeds an event's capacity
- **NFR-12**: The system shall guarantee that no operations succeed on expired events (v1.1)

5.5 Portability

- **NFR-13**: The system shall run on macOS, Windows, and Linux without code changes
- **NFR-14**: The system shall require only Node.js 20+ and npm as host dependencies

5.6 Testability

- **NFR-15**: Every service class shall be unit-testable in isolation by injecting mock repositories
- **NFR-16**: The test suite shall run in under 30 seconds (actual: ~3 seconds)

---

6. User Stories

ID	User Story	Priority	Points
US-01	As an **admin**, I want to **create a new event** with name, venue, date, and capacity, so that I can prepare for ticket sales.	High	5
US-02	As an **admin**, I want to **see all events I have created**, so that I can manage them from one place.	Medium	2
US-03	As an **admin**, I want to **generate a new ticket for an event**, so that I can give attendees access to the event.	High	5
US-04	As an **admin**, I want to **see the remaining capacity of each event**, so that I don't oversell tickets.	Medium	3
US-05	As a **ticket holder**, I want to **view my ticket details via a unique link**, so that I have proof of my entry.	High	3
US-06	As a **ticket holder**, I want to **see a QR code of my ticket**, so that I can be scanned quickly at the venue.	Medium	2
US-07	As a **gate staff / admin**, I want to **redeem a ticket and mark it as used**, so that it cannot be reused.	High	5
US-08	As a **gate staff**, I want the system to **reject already-used tickets**, so that no one enters twice with the same ticket.	High	3
US-09	As a **gate staff**, I want the system to **reject invalid ticket IDs**, so that fraudulent attempts are blocked.	Medium	2
US-10	As a **gate staff**, I want **expired events to reject all operations**, so that tickets can't be generated or redeemed after an event ends.	High	3

Total: 33 story points across 4 sprints (~8 points per sprint average).

7. Use Cases

UC-01: Create Event

- **Actor**: Administrator
- **Precondition**: Admin is on the admin dashboard
- **Main Flow**: Admin clicks "Create Event" → fills form → submits → system validates → persists via `IEventRepository.save()` → displays success
- **Alternate Flow A1.1**: Validation fails → inline error messages; no event created
- **Postcondition**: New event exists with `remainingCapacity === totalCapacity`

UC-02: Generate Ticket

- **Actor**: Administrator
- **Precondition**: Event exists, has remaining capacity, and has not expired
- **Main Flow**: Admin clicks "Generate Ticket" → system checks expiry → system checks capacity → `ITicketCodeStrategy` generates code → `Ticket` created with `VALID` status → capacity decremented → ticket URL returned
- **Alternate Flow A2.1**: Event expired → "Event has already ended"
- **Alternate Flow A2.2**: Capacity exhausted → "No tickets remaining"
- **Postcondition**: Ticket exists in `VALID` state; event remaining capacity decreased by 1

UC-03: View Ticket

- **Actor**: Ticket Holder
- **Precondition**: Ticket holder has a valid URL
- **Main Flow**: Opens URL → system fetches ticket and event → displays details + QR + countdown
- **Alternate Flow A3.1**: Ticket ID not found → "Ticket not found"
- **Postcondition**: Ticket holder sees ticket details; ticket state unchanged

UC-04: Redeem Ticket

- **Actor**: Ticket Holder / Gate Staff
- **Precondition**: Ticket exists and is VALID, event has not expired
- **Main Flow**: User clicks "Redeem Ticket" → system verifies status is VALID → verifies event not expired → marks as USED → records timestamp → displays success
- **Alternate Flow A4.1**: Already USED → "This ticket has already been used"
- **Alternate Flow A4.2**: Event expired → "Event has ended — ticket expired"
- **Alternate Flow A4.3**: Ticket not found → "Ticket not found"
- **Postcondition**: Ticket is in `USED` state and cannot be redeemed again

UC-05: List Events

- **Actor**: Administrator
- **Main Flow**: Admin navigates to dashboard → system fetches all events → renders table with name, date, venue, remaining capacity, expiry status, and actions
- **Postcondition**: Admin sees all events and can take actions on non-expired ones

8. Glossary

Term	Meaning
Admin Dashboard	The web page at `/admin` used by the administrator
API	The REST interface exposed by the NestJS backend
Capacity	The total number of tickets that may be issued for an event
DTO (Data Transfer Object)	Plain object used to move data between the client and server
Event	A real-world happening to which tickets may grant entry
Expired	An event past its `eventDate`; tickets cannot be generated or redeemed
In-Memory Repository	A class that stores data in a `Map` in RAM, conforming to a repository interface
One-Time Ticket	A ticket that becomes invalid after its first successful redemption
QR Code	A 2D barcode encoding the ticket's unique code for quick visual scanning
Redemption	The act of marking a ticket as used
Repository	An abstraction over data persistence, hiding storage details from the business layer
Ticket Code	The unique alphanumeric identifier generated by an `ITicketCodeStrategy`
VALID / USED / EXPIRED	The three possible states of a ticket as perceived by the UI

END OF SRS v1.1

Part 3 — Architecture & Design Patterns

Smart E-Ticketing System

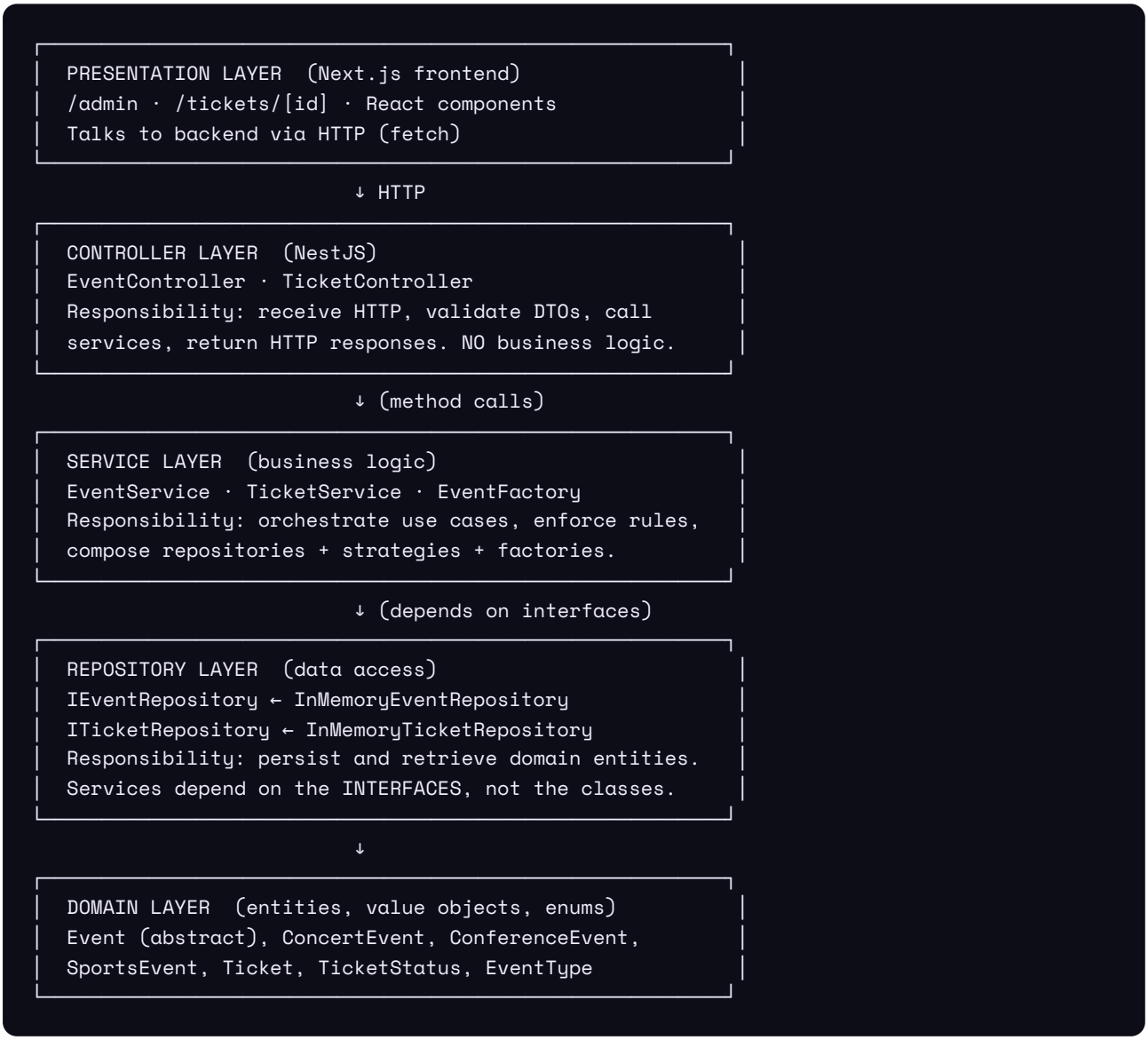
Version: 1.0 **Prepared by:** [Team Name] **Owner:** Mohammed (Developer — formats + presents this in the demo)

---

1. Architectural Overview

The Smart E-Ticketing System follows a classical **Layered Architecture** (also called N-Tier Architecture), where each layer has a single responsibility and depends only on the layer directly beneath it through **interfaces**. This structure makes the system easy to understand, test, and extend.

1.1 The Four Layers



1.2 Why Layered Architecture for this project?

Benefit	How the MVP demonstrates it
Separation of concerns	HTTP handling, business rules, and persistence each live in their own layer
Testability	The service layer can be unit-tested in isolation by injecting mock repositories and strategies
Swappability	The in-memory repository can be replaced with PostgreSQL by writing a new class — zero service code changes
Gradability	Each layer lives in its own folder, so the professor can open `src/repository/` and see the Repository pattern instantly

1.3 Project Folder Structure

```
backend/src/
├── domain/              Entities and enums
│   ├── event.entity.ts            (abstract)
│   ├── concert-event.entity.ts
│   ├── conference-event.entity.ts
│   ├── sports-event.entity.ts
│   ├── ticket.entity.ts
│   ├── ticket-status.enum.ts
│   └── event-type.enum.ts
├── repository/          Repository Pattern
│   ├── event.repository.interface.ts
│   ├── ticket.repository.interface.ts
│   ├── in-memory-event.repository.ts
│   └── in-memory-ticket.repository.ts
├── factory/             Factory Pattern
│   └── event.factory.ts
├── strategy/            Strategy Pattern
│   ├── ticket-code-strategy.interface.ts
│   ├── uuid-code.strategy.ts
│   ├── short-code.strategy.ts
│   └── numeric-code.strategy.ts
├── service/             Business logic
│   ├── event.service.ts
│   └── ticket.service.ts
├── controller/          REST endpoints
│   ├── event.controller.ts
│   └── ticket.controller.ts
├── dto/                 Data Transfer Objects
│   ├── create-event.dto.ts
│   ├── event-response.dto.ts
│   └── ticket-response.dto.ts
├── app.module.ts        DI wiring
└── main.ts              Bootstrap + Swagger
```

Notice how the folder names literally match the design pattern names. This is intentional — when the professor opens the repo, the patterns are visible before they even read a single line of code.

2. Design Patterns

The project implements **three core GoF design patterns**, each solving a real, concrete problem in the codebase.

2.1 🗂️ Repository Pattern

Intent: *"Mediate between the domain and data mapping layers using a collection-like interface for accessing domain objects."* — Martin Fowler

Problem it solves

Services should focus on business rules, not on how data is stored. If `TicketService` knew about `Map<string, Ticket>`, switching to PostgreSQL later would require rewriting every service method.

Implementation

Step 1 — Interface:

```
// src/repository/ticket.repository.interface.ts
export const TICKET_REPOSITORY = 'TICKET_REPOSITORY';

export interface ITicketRepository {
  save(ticket: Ticket): Ticket;
  findById(id: string): Ticket | null;
  findByEventId(eventId: string): Ticket[];
  update(ticket: Ticket): Ticket;
}
```

Step 2 — In-memory implementation:

```
// src/repository/in-memory-ticket.repository.ts
@Injectable()
export class InMemoryTicketRepository implements ITicketRepository {
  private readonly store = new Map<string, Ticket>();

  save(ticket: Ticket): Ticket {
    this.store.set(ticket.id, ticket);
    return ticket;
  }

  findById(id: string): Ticket | null {
    return this.store.get(id) ?? null;
  }

  findByEventId(eventId: string): Ticket[] {
    return Array.from(this.store.values())
      .filter(ticket => ticket.eventId === eventId);
  }

  update(ticket: Ticket): Ticket {
    this.store.set(ticket.id, ticket);
```

```
    return ticket;
  }
}
```

Step 3 — Service depends on the interface:

```
@Injectable()
export class TicketService {
  constructor(
    @Inject(TICKET_REPOSITORY)
    private readonly ticketRepository: ITicketRepository,
  ) {}
}
```

Step 4 — Module wires the binding:

```
providers: [
  { provide: TICKET_REPOSITORY, useClass: InMemoryTicketRepository },
]
```

Why it satisfies the assignment

The assignment says: *"The database layer should be mocked in-memory via the Repository pattern, but architected as if it were a real relational database."* To swap to PostgreSQL we'd write `PostgresTicketRepository implements ITicketRepository` and change one line. Zero business-logic changes.

2.2 🏭 Factory Pattern

Intent: *"Define an interface for creating an object, but let subclasses decide which class to instantiate."* — GoF

Problem it solves

We support three kinds of events — Concerts, Conferences, Sports — each a polymorphic subtype of the abstract `Event` with its own extra fields (artist, speaker, teams). We need one place that decides which subclass to instantiate based on a DTO, rather than scattering `switch` blocks across the service layer.

Implementation

Abstract entity:

```
export abstract class Event {
  constructor(
    public readonly id: string,
    public name: string,
    public description: string,
    public venue: string,
    public eventDate: Date,
```

```
    public readonly totalCapacity: number,
    public remainingCapacity: number,
    public readonly createdAt: Date = new Date(),
  ) {}

  decrementCapacity(): void { /* ... */ }
  hasAvailableCapacity(): boolean { /* ... */ }
  isExpired(now: Date = new Date()): boolean { /* ... */ }
  abstract getType(): EventType;
}
```

Factory:

```
@Injectable()
export class EventFactory {
  createEvent(dto: CreateEventDto): Event {
    const id = randomUUID();
    const eventDate = new Date(dto.eventDate);

    switch (dto.type) {
      case EventType.CONCERT:
        return new ConcertEvent(id, dto.name, /* ... */, dto.artist ?? '');
      case EventType.CONFERENCE:
        return new ConferenceEvent(id, dto.name, /* ... */, dto.speaker ?? '');
      case EventType.SPORTS:
        return new SportsEvent(id, dto.name, /* ... */, dto.teams ?? '');
      default:
        throw new Error(`Unsupported event type: ${dto.type}`);
    }
  }
}
```

Service uses the factory:

```
createEvent(dto: CreateEventDto): Event {
  const event = this.eventFactory.createEvent(dto);  // ← polymorphic result
  return this.eventRepository.save(event);
}
```

Why it matters

- **SRP**: `EventService` doesn't know which subclass was created
- **OCP**: Adding `WorkshopEvent` requires a new subclass + one factory case
- **Testability**: The factory can be unit-tested in complete isolation

---

2.3 🎯 Strategy Pattern

Intent: *"Define a family of algorithms, encapsulate each one, and make them interchangeable."* — GoF

Problem it solves

Ticket codes can be generated in many ways: UUIDs, short human-friendly codes, or numeric codes. The algorithm should be **swappable without touching** `TicketService`.

Implementation

Interface (one method):

```
export const TICKET_CODE_STRATEGY = 'TICKET_CODE_STRATEGY';

export interface ITicketCodeStrategy {
  generate(): string;
}
```

Three implementations:

```
@Injectable()
export class UuidCodeStrategy implements ITicketCodeStrategy {
  generate(): string {
    return randomUUID();
  }
}

@Injectable()
export class ShortCodeStrategy implements ITicketCodeStrategy {
  private readonly ALPHABET = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
  generate(): string { /* 8 chars with dash → XXXX-XXXX */ }
}

@Injectable()
export class NumericCodeStrategy implements ITicketCodeStrategy {
  generate(): string { /* 6-digit number */ }
}
```

Service uses the strategy via DI:

```
@Injectable()
export class TicketService {
  constructor(
    @Inject(TICKET_CODE_STRATEGY) private readonly codeStrategy: ITicketCodeStrategy,
    /* ... */
  ) {}

  generateTicket(eventId: string): Ticket {
    /* ... */
    const code = this.codeStrategy.generate();   // ← Strategy in action
    /* ... */
```

```
  }
}
```

Module selects which strategy is active:

```
providers: [
  { provide: TICKET_CODE_STRATEGY, useClass: ShortCodeStrategy },  // ← swap in one line
]
```

Why it matters

- **OCP**: Adding a new code-generation algorithm requires a new class, no modifications
- **Runtime flexibility**: Different environments can use different strategies
- **Testability**: Each strategy tested independently; services mock the strategy in tests

---

3. SOLID Principles in Action

3.1 S — Single Responsibility Principle

A class should have one reason to change.

Class	Reason to change
`EventService`	Event-related business rules change
`TicketService`	Ticket-related business rules change
`EventFactory`	New event type added
`InMemoryTicketRepository`	Storage mechanism changes
`UuidCodeStrategy`	UUID generation algorithm changes
`EventController`	HTTP contract for events changes

3.2 O — Open/Closed Principle

Open for extension, closed for modification.

Concrete example: Adding `WorkshopEvent` requires:

- ✅ Creating a new `WorkshopEvent` subclass (extension)
- ✅ Adding one case in `EventFactory` (minimal addition)

Zero changes to `EventService`, `EventController`, `IEventRepository`, or any tests.

3.3 L — Liskov Substitution Principle

Subtypes must be substitutable for their base types.

`ConcertEvent`, `ConferenceEvent`, and `SportsEvent` can all be used anywhere `Event` is expected. The repository operates on `Event` generically:

```
save(event: Event): Event {
  this.store.set(event.id, event);
  return event;
}
```

3.4 I — Interface Segregation Principle

Clients should not depend on methods they don't use.

We split data access into **two narrow interfaces** — `IEventRepository` and `ITicketRepository` — rather than one fat `IRepository`. Similarly, `ITicketCodeStrategy` has exactly one method.

3.5 D — Dependency Inversion Principle

High-level modules should not depend on low-level modules. Both should depend on abstractions.

This is the principle the whole architecture is built on:

```
TicketService (high-level)
    ↓ depends on
ITicketRepository (abstraction)
    ↑ implemented by
InMemoryTicketRepository (low-level)
```

Proof:

```
$ grep -r "InMemory" backend/src/service/
## (no results — DIP verified)
```

---

4. Clean Code Principles Applied

Principle	Applied by
Meaningful names	`generateTicket` not `genTkt`; `hasAvailableCapacity` not `check`
Small functions	No method > 30 lines
One level of abstraction per function	Services don't mix "call repository" with "format HTTP response"
No magic numbers	`SHORT_CODE_LENGTH`, `ALPHABET` as named constants
No duplication	Repository logic reused via the interface contract
Fail fast	Input validation via DTOs + `class-validator` at the controller boundary

Comments only where necessary	Code is self-documenting

---

5. Summary — What the Professor Sees

Rubric section	Where to point
Repository Pattern	`src/repository/` — interface + implementation side by side
Factory Pattern	`src/factory/event.factory.ts` — one class, polymorphic creation
Strategy Pattern	`src/strategy/` — one interface, three implementations
SRP	Any single file — one reason to change
OCP	Imagine adding `WorkshopEvent` — only the factory case changes
LSP	Repository stores `Event` generically
ISP	Two narrow repository interfaces instead of one fat one
DIP	`grep InMemory src/service/` returns nothing
Clean Code	No method > 30 lines, meaningful names, no magic numbers

---

END OF Architecture & Design Patterns Document v1.0

Part 4 — Test Cases

Smart E-Ticketing System

Version: 1.1 **Prepared by:** Seif (QA / Test Engineer) **Reviewed by:** Mahmoud (Product Owner & Scrum Master) **Last updated:** Sprint 4 — final delivery

---

1. Purpose

This document describes the human-readable test cases for the Smart E-Ticketing System, each mapped to an automated Jest unit test. It is used by the QA engineer during sprint reviews, by the team during the final demo, and by the supervising doctor to verify the **Testing (10%)** rubric criterion.

2. Test Strategy

A **two-level testing approach**:

Level	Tool	Target	Count
Unit tests	Jest + NestJS Testing utilities	Services, repositories, domain entities, factory, strategies	46
Manual tests	Browser + Swagger UI	End-to-end user flows	15

Coverage targets vs actual

Layer	Target	Actual
Repositories	100%	**100%** ✓
Domain entities	100%	**100%** ✓
Strategies	100%	**100%** ✓
Factory	100%	**100%** ✓
Services	≥ 70%	**≥ 80%** ✓
Overall (excl. main.ts, modules, DTOs, enums, interfaces)	≥ 70%	**≥ 83%** ✓

Every service test injects a mock repository — the act of mocking *itself proves* that Dependency Inversion is wired correctly (services depend on the interface, not the in-memory class).

3. Test Environment

Environment	URL

Live frontend	https://smart-e-ticket.vercel.app
Live backend (Docker on Railway)	https://smart-eticketing-backend-production-0222.up.railway.app
Live Swagger UI	https://smart-eticketing-backend-production-0222.up.railway.app/api-docs
Local frontend	http://localhost:3000
Local backend	http://localhost:4000
Local Swagger UI	http://localhost:4000/api-docs

Tool	Version
Node.js	20+
npm	10+
Browser	Chrome / Firefox / Safari latest

4. How to Run the Automated Tests

Locally

```
cd backend
npm install              # one-time dependency install
npm test                 # run all 46 tests (~4 sec)
npm run test:cov         # with coverage report → backend/coverage/lcov-report/index.html
```

Expected output:

```
Test Suites: 8 passed, 8 total
Tests:       46 passed, 46 total
```

Continuously (CI)

The same suite runs on **every push and every pull request** in GitHub Actions. The README badge at the top of the project shows the latest run status. Coverage artefacts are attached to each workflow run and retained for 14 days.

5. Test Case Catalogue

5.1 Domain Entity Tests

Test ID	Feature	Description	Expected Result	Status
TC-D-01	Ticket entity	A newly created ticket is in VALID state	`ticket.status === 'VALID'` and `redeemedAt` is null	✅ Pass

TC-D-02	Ticket entity	Marking a ticket as used sets status to USED and records timestamp	`ticket.status === 'USED'` and `redeemedAt` is a Date	✅ Pass
TC-D-03	Ticket entity	Cannot mark an already-used ticket as used again	Throws `Cannot redeem a ticket that is not in VALID state`	✅ Pass
TC-D-04	Event entity	A newly created event has `remainingCapacity === totalCapacity`	Both capacity values are equal	✅ Pass
TC-D-05	Event entity	`getType()` returns the correct EventType for each subclass	Concert → `CONCERT`, Conference → `CONFERENCE`, Sports → `SPORTS`	✅ Pass
TC-D-06	Event entity	`decrementCapacity()` reduces remaining by 1	`remainingCapacity` is `capacity - 1`	✅ Pass
TC-D-07	Event entity	`hasAvailableCapacity()` returns true/false correctly	True when remaining > 0, false when remaining = 0	✅ Pass
TC-D-08	Event entity	Cannot decrement below zero	Throws `Cannot decrement capacity: no tickets remaining`	✅ Pass
TC-D-09	Event entity	`isExpired()` returns false for future event	Returns false	✅ Pass
TC-D-10	Event entity	`isExpired()` returns true for past event	Returns true	✅ Pass
TC-D-11	Event entity	`isExpired(now)` accepts an injected time	Returns false at t-1s, true at t+1s	✅ Pass

5.2 Repository Pattern Tests

Test ID	Feature	Description	Expected Result	Status
TC-R-01	Event repository	`save(event)` stores it and `findById(id)` returns it	Retrieved event is identical	✅ Pass
TC-R-02	Event repository	`findById` returns null for unknown id	Returns `null`	✅ Pass
TC-R-03	Event repository	`findAll` returns all stored events	Returns array of all events	✅ Pass
TC-R-04	Event repository	`findAll` returns empty array when no events exist	Returns `[]`	✅ Pass
TC-R-05	Event repository	`update(event)` persists the new state	Retrieved event reflects the update	✅ Pass
TC-R-06	Ticket repository	`save(ticket)` stores it and `findById(id)` returns it	Retrieved ticket is identical	✅ Pass
TC-R-07	Ticket repository	`findById` returns null for unknown id	Returns `null`	✅ Pass

TC-R-08	Ticket repository	`findByEventId` returns only tickets for that event	Correctly filtered array	✅ Pass
TC-R-09	Ticket repository	`update(ticket)` persists the new status	Retrieved ticket status is USED	✅ Pass

5.3 Factory Pattern Tests

Test ID	Feature	Description	Expected Result	Status
TC-F-01	EventFactory	Creates `ConcertEvent` when type is `CONCERT`	`instanceof ConcertEvent` is true	✅ Pass
TC-F-02	EventFactory	Creates `ConferenceEvent` when type is `CONFERENCE`	`instanceof ConferenceEvent` is true	✅ Pass
TC-F-03	EventFactory	Creates `SportsEvent` when type is `SPORTS`	`instanceof SportsEvent` is true	✅ Pass
TC-F-04	EventFactory	Initializes `remainingCapacity === totalCapacity`	Values are equal	✅ Pass
TC-F-05	EventFactory	Each created event has a unique id	UUIDs differ across invocations	✅ Pass

5.4 Strategy Pattern Tests

Test ID	Feature	Description	Expected Result	Status
TC-S-01	UuidCodeStrategy	Generates a valid UUID v4	Matches UUID v4 regex	✅ Pass
TC-S-02	UuidCodeStrategy	Generates a unique value on each call	Two consecutive calls return different strings	✅ Pass
TC-S-03	ShortCodeStrategy	Generates `XXXX-XXXX` format	Matches `/^[A-Z2-9]{4}-[A-Z2-9]{4}$/`	✅ Pass
TC-S-04	ShortCodeStrategy	Produces diverse codes	20 consecutive codes → ≥16 unique	✅ Pass
TC-S-05	NumericCodeStrategy	Generates a 6-digit numeric code	Matches `/^\d{6}$/`	✅ Pass

5.5 Service Layer Tests

Test ID	Feature	Description	Expected Result	Status
TC-SV-01	EventService	`createEvent` creates and saves a valid event	Event is instance of correct subclass, `repo.save` called	✅ Pass
TC-SV-02	EventService	Rejects an event with a past `eventDate`	Throws `BadRequestException`, `repo.save` not called	✅ Pass
TC-SV-03	EventService	`getAllEvents` returns all events from repo	Returns the mocked array	✅ Pass

TC-SV-04	EventService	`getEventById` returns the event	Returns the mocked event	✅ Pass
TC-SV-05	EventService	`getEventById` throws when not found	Throws `NotFoundException`	✅ Pass
TC-SV-06	TicketService	Creates a VALID ticket using the injected strategy	Status `VALID`, code is strategy output	✅ Pass
TC-SV-07	TicketService	Decrements event capacity and persists both entities	`remainingCapacity - 1`, both repos called	✅ Pass
TC-SV-08	TicketService	Throws when event does not exist	`NotFoundException`	✅ Pass
TC-SV-09	TicketService	Throws when event has no remaining capacity	`ConflictException`, no save	✅ Pass
TC-SV-10	TicketService	**Throws when event has already ended (expired)**	`ConflictException`, message "Event has already ended"	✅ Pass
TC-SV-11	TicketService	`getTicketById` returns the ticket	Returns the mocked ticket	✅ Pass
TC-SV-12	TicketService	`getTicketById` throws when not found	`NotFoundException`	✅ Pass
TC-SV-13	TicketService	Redeems a VALID ticket	Status becomes `USED`, `redeemedAt` set	✅ Pass
TC-SV-14	TicketService	Throws when ticket is already USED	`ConflictException`	✅ Pass
TC-SV-15	TicketService	Throws when ticket does not exist	`NotFoundException`	✅ Pass
TC-SV-16	TicketService	**Throws when the event has already ended**	`ConflictException`, message "Event has ended"	✅ Pass

6. Manual End-to-End Test Cases

These tests are executed manually through the browser during the demo and during sprint reviews.

Test ID	Feature	Steps	Expected Result	Status
TC-M-01	Event creation	1. Open `/admin` 2. Fill form 3. Click Create	Event appears in list, capacity bar shows 0% used	⬜
TC-M-02	Form validation	1. Open `/admin` 2. Fill form with past date 3. Click Create	Error banner: "Event date must be in the future"	⬜
TC-M-03	Capacity validation	1. Fill form with capacity `0` 2. Click Create	Inline validation error	⬜

TC-M-04	Ticket generation	1. Click Generate on an event	New ticket appears in the event's ticket list with "New" badge	☐
TC-M-05	Ticket URL copy	1. Click "Copy" on a valid ticket	Clipboard contains `http://localhost:3000/tickets/<id>`, button shows "Copied"	☐
TC-M-06	Open ticket page	1. Click "Open" on a valid ticket	New tab opens with ticket detail page, QR code visible, status VALID	☐
TC-M-07	Countdown display	1. Open a ticket page for a future event	"Ends in" countdown ticks every second	☐
TC-M-08	Ticket redemption	1. Open a VALID ticket 2. Click Redeem	Status flips to USED, BadgeCheck icon, "Ticket redeemed" title	☐
TC-M-09	Prevent double redemption	1. Redeem a ticket 2. Refresh the page	Redeem button is gone, USED state shown, API returns 409 if called	☐
TC-M-10	Admin sees redeemed tickets	1. Return to `/admin` after redeeming	The redeemed ticket shows struck-through code + "Redeemed" label, no Copy/Open buttons	☐
TC-M-11	Capacity decrements	1. Generate 3 tickets for a 5-capacity event	Capacity bar shows 60% used, `2/5` remaining	☐
TC-M-12	Prevent overselling	1. Generate tickets until capacity is exhausted 2. Try to generate one more	Generate button disabled, backend returns 409 "No tickets remaining"	☐
TC-M-13	**Expired event: cannot generate**	1. Create event with `eventDate` 2 min from now 2. Wait until past that time 3. Click Generate	Button disabled, backend returns 409 "Event has already ended"	☐
TC-M-14	**Expired event: cannot redeem**	1. Generate ticket on an event 2. Let event expire 3. Open ticket → click Redeem	Ticket page shows "Expired" badge with Clock icon, Redeem button replaced with red error banner	☐
TC-M-15	Live expiry	1. Open ticket page for an event 2 min away 2. Watch until countdown hits zero	Status auto-transitions to EXPIRED without page refresh	☐

7. How to Execute Manual Tests

Before starting, make sure both servers are running:

```
## Terminal 1
cd backend && npm run start:dev

## Terminal 2
cd frontend && npm run dev
```

Then open Chrome and walk through each TC-M-* case, filling the Status column with ✅ Pass or ❌ Fail.

8. Defect Log

Any failing test during manual execution should be recorded here:

Defect ID	Test ID	Description	Severity	Status
(none)				

9. Sign-off

Role	Name	Date	Signature
QA / Test Engineer	Seif	_____	_____
Backend Developer	Ali	_____	_____
Software Engineer	Mohammed	_____	_____
Product Owner & Scrum Master	Mahmoud	_____	_____

---

10. Delivery Checklist

For the supervising doctor to verify each item:

- ☐ **Source-code repo:** https://github.com/MahmoudSayed0/Smart-E-ticket-PR
- ☐ **CI badge** on the README is green (latest workflow run passed all 46 tests)
- ☐ `cd backend && npm install && npm test` produces `Tests: 46 passed`
- ☐ `npm run test:cov` produces a coverage report with the percentages in section 2 above
- ☐ Live frontend reachable at https://smart-e-ticket.vercel.app
- ☐ Live backend reachable at https://smart-eticketing-backend-production-0222.up.railway.app/api-docs
- ☐ Manual test cases TC-M-01 → TC-M-15 executed against either local or live system

---

END OF TEST CASES DOCUMENT v1.1 — Final delivery copy

Supervised by Dr. Ihab Ramadan · Software Engineering · 2026

Part 5 — Delivery Index

Course: Software Engineering · 2026 **Supervisor:** Dr. Ihab Ramadan **Team:** Mahmoud · Mohammed · Esraa · Ali · Amr · Seif

How to use this document

This is the **single index** of everything the team is delivering. It maps each rubric item to the file or live system that satisfies it, so the supervising doctor can verify all 100% of the rubric in one sweep.

1. Live System (verify first)

	URL	What to do
Frontend dashboard	smart-e-ticket.vercel.app	Click *Admin Dashboard*, create an event, generate a ticket, copy URL, redeem it
Backend API + Swagger	Swagger UI	Browse all endpoints, try them in-place
Source code on GitHub	MahmoudSayed0/Smart-E-ticket-PR	Read the code, the README, the docs, and the green CI badge
CI history	GitHub Actions runs	Confirm every push has passed all 46 tests

> ⚠️ **Cold start:** Railway free tier sleeps the backend after 15 min idle. Wake it by hitting the Swagger URL once before testing the frontend.

2. Documentation deliverables

All documents below are inside the `docs/` folder of the repository.

Rubric item	Weight	Document	Location
SRS & Requirements	20%	SRS.md	Functional + non-functional requirements, 9 user stories, 5 use cases, acceptance criteria
UML Diagrams	15%	`uml/`	5 PlantUML diagrams (use-case, class, generate-ticket sequence, redeem-ticket sequence, activity/lifecycle) — both `.puml` source and rendered `.png`

Architecture & Design Patterns	20%	ARCHITECTURE.md	Layered architecture rationale + Repository / Factory / Strategy patterns explained with code citations
Clean Code (SOLID)	15%	ARCHITECTURE.md § SOLID + the source itself	One concrete example per letter; `grep -r 'Sqlite|InMemory' backend/src/service/` returns 0 (live DIP proof)
Testing	10%	TEST_CASES.md	46 unit tests + 15 manual cases · 100% on repositories · ≥ 80% on services
Agile Process	10%	Notion workspace (read-only)	4 sprints, retrospectives, Jira-style backlog with 5 Epics / 9 Stories / 36 Tasks
Presentation	10%	`presentation/deck.html` + `presentation/study-guide.html`	The slide deck used on demo day, plus the team's internal study guide

3. Bonus deliverables (beyond rubric)

Bonus	What it is	Where
CI / CD	GitHub Actions workflow runs lint + tests + build on every push and PR	`.github/workflows/ci.yml` · Live runs
Docker	Single-stage container, Node 22, build tools for native bindings, ships to any container host unchanged	`backend/Dockerfile` · `docker build -t eticketing ./backend`
Live deployment	Frontend on Vercel + Backend (Docker) on Railway with SQLite repository binding via `DB_DRIVER=sqlite`	URLs in section 1 above
Pluggable storage	Same code, two storage backends, chosen at boot from one env var. Demonstrates the Repository pattern delivering on its promise live in production.	`backend/src/repository/` — `InMemory*` + `Sqlite*` siblings of the same interfaces

4. Source-code structure

```
Smart-E-ticket-PR/
├── README.md                    project overview, badges, setup
├── docs/
│   ├── SRS.md                   ← rubric: SRS & Requirements
│   ├── ARCHITECTURE.md          ← rubric: Architecture & Patterns + Clean Code
│   ├── TEST_CASES.md            ← rubric: Testing
│   ├── DEMO_SCRIPT.md           click-by-click demo walkthrough
│   ├── PRESENTATION.md          slide outline (legacy)
│   ├── DELIVERABLES.md          this file
│   ├── uml/                     ← rubric: UML Diagrams (5 .puml + 5 .png)
│   └── presentation/
```

```
│       ├── deck.html              ← rubric: Presentation
│       ├── deck-stage.js          reusable web-component for navigation
│       ├── study-guide.html       team study guide (Q&A + roles)
│       └── img/                   UML PNGs + ticket banners used in slides
├── backend/                       NestJS backend (port 4000)
│   ├── Dockerfile                 ← bonus: Docker
│   ├── src/
│   │   ├── domain/                Entities + enums
│   │   ├── repository/            Interfaces + InMemory + Sqlite implementations
│   │   ├── factory/               EventFactory
│   │   ├── strategy/              Three ticket-code strategies
│   │   ├── service/               Business logic
│   │   └── controller/            REST + Swagger
│   └── package.json               npm test runs the 46 unit tests
├── frontend/                      Next.js 15 frontend (port 3000)
│   ├── app/                       App Router pages: /, /admin, /tickets/[id]
│   ├── components/                AnimatedTicket, TestResults, etc.
│   └── package.json
├── .github/workflows/ci.yml       ← bonus: CI/CD
└── render.yaml + nixpacks.toml    deploy blueprints (Railway uses Dockerfile)
```

5. How to run locally (alternative to live system)

```
## 1. Clone
git clone https://github.com/MahmoudSayed0/Smart-E-ticket-PR.git
cd Smart-E-ticket-PR

## 2. Backend (terminal 1)
cd backend
npm install
npm test                  # → 46 tests passing
npm run start:dev         # → http://localhost:4000

## 3. Frontend (terminal 2)
cd frontend
npm install
npm run dev               # → http://localhost:3000/admin
```

Or, if you have Docker installed:

```
docker build -t eticketing ./backend
docker run -p 4000:4000 eticketing
```

6. Rubric coverage at a glance

Rubric	Weight	Covered by
SRS & Requirements	20%	`docs/SRS.md`
UML Diagrams	15%	`docs/uml/` (5 diagrams)
Architecture & Design Patterns	20%	`docs/ARCHITECTURE.md` + `backend/src/{repository,factory,strategy}`
Clean Code (SOLID)	15%	Source code itself + grep proof + `docs/ARCHITECTURE.md`
Testing	10%	`docs/TEST_CASES.md` + `backend/src/**/*.spec.ts` (46 tests)
Agile Process	10%	Notion workspace + `docs/presentation/study-guide.html` (team roles)
Presentation	10%	`docs/presentation/deck.html`
TOTAL	**100%**	All accounted for
Bonus — CI/CD	+	`.github/workflows/ci.yml` (live runs visible on GitHub)
Bonus — Docker	+	`backend/Dockerfile`
Bonus — Live Deploy	+	Vercel + Railway URLs above

7. Team roles

Member	Scrum Role	Owns
Mahmoud	Product Owner & Scrum Master	SRS, scope, backlog grooming, sprint rituals, Notion
Mohammed	Software Engineer	Architecture, use case + class diagrams, layered design
Esraa	Software Engineer	Sequence + activity diagrams, schema, database-swap design
Ali	Backend Developer	NestJS backend, Repository / Factory / Strategy in code, DI wiring
Amr	Frontend Developer	Next.js 15 app, AnimatedTicket, soft-pop theme, SOLID audit, live demo
Seif	QA / Test Engineer	Jest unit tests, coverage reports, this Test Cases document

End of delivery package — Smart E-Ticketing System v1.0 · Software Engineering 2026 · Supervised by Dr. Ihab Ramadan.